# Guía del equipo: el proyecto explicado fácil

Esta guía explica **todo lo que hay hoy en el repositorio** con palabras sencillas. No hace falta saber biología. Al final hay una lista de **preguntas probables de la sustentación** con su respuesta corta.

> Regla de oro de la sustentación: el profesor puede preguntarle a **cualquiera** sobre **cualquier** componente, no solo sobre los que programó. Léanla completa.

---

## 1. ¿De qué trata el proyecto?

### LA ANALOGÍA

Imaginen un libro gigante escrito solo con 4 letras: **A, C, G, T**. Ese libro es el **genoma**. Alguien arrancó miles de pedacitos de 50 letras de ese libro (las **lecturas**), pero al copiarlos cometió algunos errores:

- cambió una letra por otra (**sustitución**);
- metió una letra de más (**inserción**);
- se saltó una letra (**borrado**).

**Nuestro trabajo:** para cada pedacito, encontrar **en qué página y en qué renglón del libro estaba**, aunque tenga errores. Eso se llama **alinear lecturas**.

### POR QUÉ NO BUSCAMOS “A LO BRUTO”

Comparar cada lectura contra todo el genoma letra por letra es lentísimo. La idea inteligente es la del **índice de un libro**:

1. Antes de buscar nada, se arma un **índice** con todas las “palabras cortas” del genoma y dónde aparece cada una.
2. Para cada lectura, se toman sus palabras cortas y se buscan en el índice. Eso da **pistas** de dónde podría estar.

3. Solo en esa zona pequeña (la **región candidata**) se hace la comparación cuidadosa, la que tolera errores.

Esas “palabras cortas” se llaman **k-mers** y las pistas se llaman **semillas**.

### LOS DATOS SON INVENTADOS

El enunciado exige datos **sintéticos** generados por nosotros con **semilla fija**. Semilla fija quiere decir que, si se corre el generador dos veces, sale exactamente lo mismo, y así los experimentos se pueden repetir.

---

# 2. Conceptos básicos

### ¿QUÉ ES UN K-MER?

Es un pedazo de **k letras seguidas**. Con el genoma `ACGTAC` y k = 3, los k-mers son:

| Posición | k-mer |
|---|---|
| 0 | ACG |
| 1 | CGT |
| 2 | GTA |
| 3 | TAC |

Un genoma de n letras tiene **n − k + 1** k-mers. Guardamos cada uno junto con su posición: `(k-mer, posición)`.

### ¿POR QUÉ CONVERTIMOS LOS K-MERS EN NÚMEROS? (CODIFICACIÓN DE 2 BITS)

Comparar textos es lento: para saber si `ACGTTGCA` < `ACGTTGCT` hay que mirar letra por letra. Por eso convertimos cada letra en un número de 2 bits:

| Letra | Número | Bits |
|---|---|---|
| A | 0 | 00 |
| C | 1 | 01 |
| G | 2 | 10 |
| T | 3 | 11 |

y pegamos los bits. Por ejemplo, `ACG` queda `00 01 10` = **6**.

Ventajas:

- **Comparar dos k-mers es comparar dos números enteros**: una sola operación.
- El orden de los números es **el mismo orden alfabético** de los textos (A < C < G < T). Ordenar números es lo mismo que ordenar los textos.
- Con k ≤ 31 el número cabe en 64 bits. Eso servirá en C5, donde los guardamos en "disco".

**Truco de la ventana deslizante:** para pasar del k-mer de la posición i al de la posición i+1 no hay que recalcular todo. Se corre el número 2 bits a la izquierda, se agrega la letra nueva al final y se borra la letra que salió por la izquierda. Así, sacar todos los k-mers cuesta **Θ(n)** en vez de Θ(n·k).

Archivo: `comun/kmers.py`.

## NOTACIÓN O, Ω Y Θ (REPASO RÁPIDO)

| Símbolo | Significa | Ejemplo en palabras |
|---|---|---|
| O(f) | "crece **a lo sumo** como f" (techo) | "nunca será peor que n²" |
| Ω(f) | "crece **al menos** como f" (piso) | "ningún algoritmo puede ser más rápido que n log n" |
| Θ(f) | "crece **exactamente** como f" (techo y piso a la vez) | "siempre es n log n" |

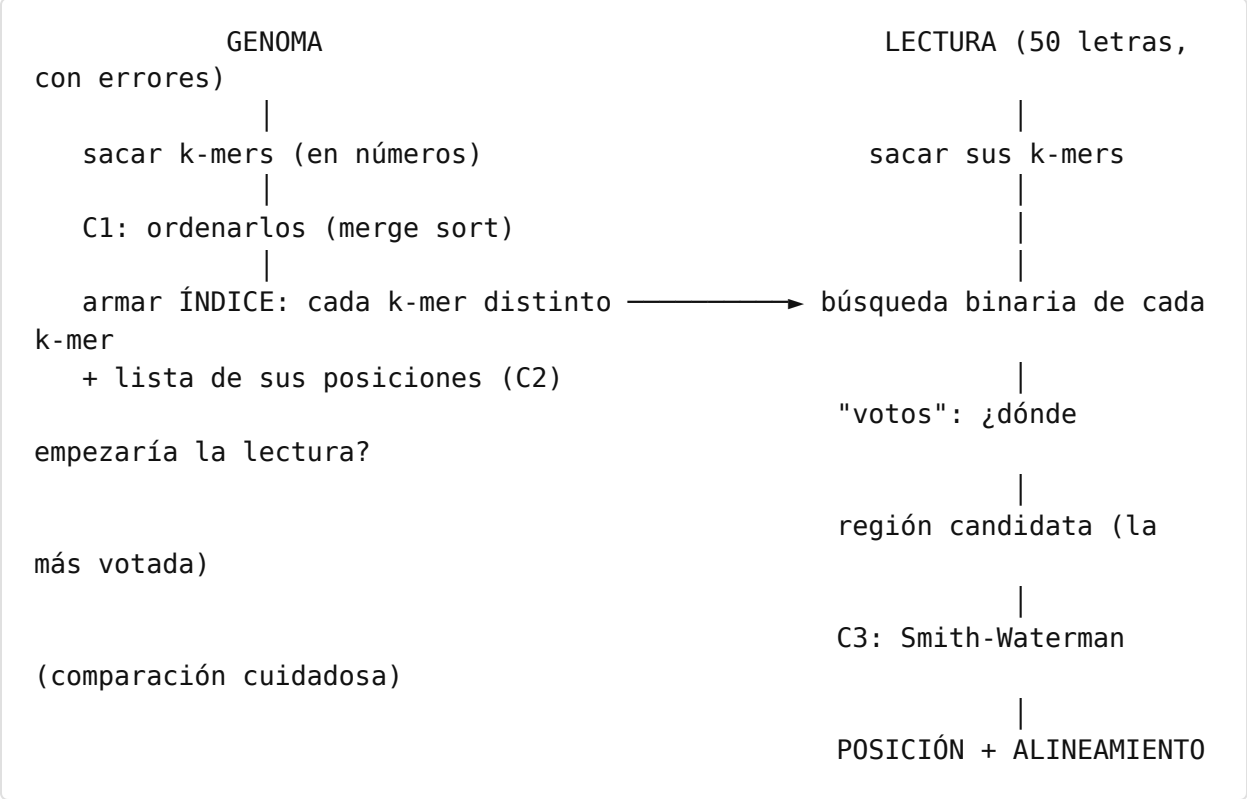
**Mejor, peor y caso esperado:** son tres preguntas distintas.

- **Mejor caso:** ¿cuánto tarda con los datos más favorables?
- **Peor caso:** ¿y con los más desfavorables?
- **Caso esperado:** ¿y en promedio, con datos al azar?

Siempre hay que decir **qué datos provocan** cada caso.

---

# 3. El recorrido completo (pipeline)




Los otros tres componentes:

- **C4:** otra forma de ordenar (quicksort aleatorizado), para compararla con C1. También ordena cada pedazo en C5.
- **C5:** construir el índice de un genoma que **no cabe en memoria** (ordenamiento externo sobre un disco simulado).
- **C6:** repartir las lecturas entre varios procesos que trabajan **en paralelo** (MapReduce).

---

# 4. C1: Merge sort (ordenar los k-mers)

## PARA QUÉ LO USAMOS

Para construir el índice necesitamos los k-mers **ordenados**. Así los k-mers iguales quedan juntos y luego se pueden buscar rápido con búsqueda binaria.

## CÓMO FUNCIONA (DIVIDE Y VENCERÁS)

1. **Dividir:** partir la lista en dos mitades.
2. **Vencer:** ordenar cada mitad (llamando a merge sort otra vez, es decir, recursión).
3. **Combinar: mezclar** las dos mitades ya ordenadas en una sola.

Mezclar es como juntar dos filas de personas ordenadas por estatura: se mira a la primera persona de cada fila, pasa la más baja y se repite. Cada paso hace **una comparación**.

```
[5 2 8 1]  →  [5 2] [8 1]  →  [5][2] [8][1]  →  [2 5] [1 8]  →  [1 2 5 8]
```

## LA RECURRENCIA, LEÍDA DEL CÓDIGO

```
def merge_sort(a):
    if len(a) <= 1: return a                  # caso base: 0(1)
    mid = len(a) // 2
    return mezclar(merge_sort(a[:mid]),       # T(n/2)
                   merge_sort(a[mid:]))       # T(n/2)   + mezclar: 0(n)
```

```
T(n) = 2·T(n/2) + 0(n)
```

En palabras: “ordenar n cuesta lo de ordenar dos mitades más una mezcla lineal”.

## TEOREMA MAESTRO (CÓMO RESOLVERLA)

Para T(n) = a·T(n/b) + f(n) se compara f(n) con n^(log_b a):

- a = 2 (dos llamadas recursivas), b = 2 (cada una con la mitad) y f(n) = n (la mezcla).

- n^($\log_2$ 2) = $n^1$ = **n**.
- f(n) = n es **igual** a n^(log_b a). Ese es el **caso 2** del Teorema Maestro.
- Resultado: **T(n) = Θ(n log n)**.

**Intuición con un dibujo (árbol de recursión):** cada nivel del árbol hace en total n de trabajo (mezclar todo), y hay $\log_2$ n niveles, porque partir a la mitad repetidamente hasta llegar a 1 toma $\log_2$ n pasos. Total: n · $\log_2$ n.

## MEJOR, PEOR Y CASO ESPERADO

El **tiempo** es Θ(n log n) siempre: merge sort parte y mezcla igual sin importar los datos. Lo que cambia es el **número de comparaciones**:

| Caso | Datos | Comparaciones |
|---|---|---|
| Mejor | lista ya ordenada (una mitad “se acaba” pronto en cada mezcla) | $(n/2)\cdot\log_2 n$ |
| Peor | mitades que se intercalan perfectamente | $n\cdot\log_2 n - n + 1$ |
| Promedio | orden al azar | $\approx n\cdot\log_2 n - 1{,}25\cdot n$ |

## LA COTA INFERIOR Ω(N LOG N): EL ÁRBOL DE DECISIÓN

**Pregunta:** ¿podría existir un algoritmo que ordene comparando y sea más rápido que n log n? **Respuesta:** no. Este es el argumento:

1. Cualquier algoritmo que ordena comparando se puede dibujar como un **árbol de preguntas sí/no** (“¿x ≤ y?”).
2. Cada **hoja** del árbol es un posible resultado (un orden final). Como hay **n!** formas de ordenar n elementos, el árbol necesita **al menos n! hojas**.
3. Un árbol binario de altura h tiene como máximo $2^h$ hojas. Entonces $2^h \geq n!$, es decir **$h \geq \log_2(n!)$**.
4. Y $\log_2(n!)$ es aproximadamente $n\cdot\log_2 n$. Entonces la altura, que es el número de comparaciones en el peor caso, es **Ω(n log n)**.

Conclusión: **merge sort es óptimo** (no se puede hacer mejor asintóticamente comparando).

> Detalle fino que puede preguntar el profe: la cota es para el **peor caso** (y para el promedio), **no para toda entrada**. Con una lista ya ordenada, merge sort hace $(n/2)\cdot\log_2 n$ comparaciones, que es *menos* que $\log_2(n!)$. Hay un test que lo demuestra.

**Contador de comparaciones:** el código cuenta cada comparación (`ms.comparaciones`) para comparar la teoría con la práctica.

Archivos: `c1_mergesort/merge_sort.py`, `tests/test_c1_mergesort.py`.

---

# 5. C2: Arreglo dinámico (listas de posiciones)

### PARA QUÉ LO USAMOS

En el índice, cada k-mer tiene una **lista de posiciones** donde aparece. No sabemos de antemano cuántas posiciones tendrá cada uno, así que necesitamos una lista que **crezca sola**.

### CÓMO FUNCIONA

- El arreglo tiene una **capacidad** (casillas reservadas) y un **tamaño** (casillas usadas).
- Al agregar un elemento, si hay espacio, se escribe y ya está: **costo 1**.
- Si está **lleno**, se reserva un arreglo del **doble** de tamaño, se **copian** todos los elementos y luego se escribe el nuevo. Esa operación es cara: cuesta n.

```
capacidad 1: [a]
capacidad 2: [a b]           ← al agregar b se copió a  (1 copia)
capacidad 4: [a b c _]       ← al agregar c se copiaron a, b  (2 copias)
capacidad 8: [a b c d e _ _ _]  ← al agregar e se copiaron 4
```

### ANÁLISIS AMORTIZADO: EL MÉTODO CONTABLE (LA ANALOGÍA DE LA ALCANCÍA)

**Pregunta:** si a veces agregar cuesta 1 y a veces cuesta n, ¿cuánto cuesta “en promedio” a lo largo de muchas operaciones? (Esto **no** es promedio sobre datos al azar; es el reparto del costo a lo largo de una secuencia.)

**Método contable:**

1. **Cobramos 3 créditos** por cada `agregar`, aunque cueste menos.
2. 1 crédito paga la **escritura** del elemento.
3. Los otros 2 se guardan **en la alcancía**, “pegados” a ese elemento.
4. Cuando toca duplicar y copiar c elementos, la alcancía **ya tiene justo c créditos**: los c/2 elementos que llegaron desde la última duplicación dejaron 2 cada uno, y $c/2 \cdot 2 = c$.
5. La alcancía **nunca queda en negativo**.

Entonces el costo real total de n operaciones es **≤ 3n**, y el **costo amortizado** por operación es **O(1)**, aunque una operación individual pueda costar n.

**Evidencia en el código:** la clase lleva la alcancía en `saldo` y guarda el valor más bajo que tuvo (`saldo_minimo`). El test comprueba que nunca baja de 0.

| Operación | Peor caso (una sola) | Amortizado |
|---|---|---|
| agregar | Θ(n) (cuando duplica) | **O(1)** |
| leer a[i] | Θ(1) | Θ(1) |

Archivos: `c2_arreglo_dinamico/arreglo_dinamico.py`, `tests/test_c2_arreglo_dinamico.py`.

---

# 6. El índice y la búsqueda binaria (une C1 y C2)

**Construir el índice:**

1. Sacar todos los `(k-mer, posición)` del genoma.
2. Ordenarlos con **C1**.
3. Recorrer la lista ordenada: cada k-mer distinto se guarda una vez en `claves`, y sus posiciones se agregan a un **arreglo dinámico (C2)**.

**Buscar un k-mer: búsqueda binaria.** Es como buscar una palabra en el diccionario: se abre por la mitad, se decide si la palabra está antes o después, y se descarta la otra mitad. Cada paso parte el problema en 2, así que tarda **Θ(log n)**.

**Votos:** si el k-mer que está en el desplazamiento o de la lectura aparece en la posición p del genoma, eso es un voto a que “la lectura empieza en p − o”. La posición con más votos es la **región candidata**.

Archivos: `comun/indice.py`, `comun/alineador.py`.

---

# 7. C3: Smith-Waterman (la comparación cuidadosa)

### PARA QUÉ LO USAMOS

La región candidata nos dice **más o menos** dónde está la lectura. Smith-Waterman encuentra **exactamente** cómo encaja, aunque haya letras cambiadas, sobrantes o faltantes.

### LOS PUNTAJES

Se pone la lectura encima de la región, columna por columna:

```
lectura:  A C G T - A C G
región:   A C G T T A C G
          + + + + − + + +
```

| Columna | Significa | Puntaje |
|---|---|---|
| letras iguales | coincide | **+2** |
| letras distintas | mutación o error | **−1** |
| hueco `-` | sobra o falta una letra | **−2** |

En el ejemplo: 7 coincidencias × 2 − 1 hueco × 2 = **12**. Smith-Waterman encuentra el acomodo con el **mayor puntaje posible**.

**¿Por qué “local”?** Porque no obligamos a alinear todo con todo. Si un tramo empieza a restar demasiado, el algoritmo lo **descarta y vuelve a empezar desde 0**. Así encuentra el pedazo que de verdad se parece, aunque la región tenga letras de más a los lados.

## PROGRAMACIÓN DINÁMICA: LA TABLA

Se arma una **tabla H** con la lectura en las filas y la región en las columnas. Cada casilla H[i][j] responde: *"¿cuál es el mejor puntaje de un alineamiento que **termina** justo en la letra i de la lectura y la letra j de la región?"*

Cada casilla se calcula mirando **solo 3 vecinas** (diagonal, arriba e izquierda) y quedándose con la mejor de 4 opciones:

```
              ┌ 0                                  empezar de cero
H[i][j] = max │ H[i-1][j-1] + (+2 ó −1)           diagonal: emparejar
las dos letras
              │ H[i-1][j]   − 2                    arriba: hueco en la
región
              └ H[i][j-1]   − 2                    izquierda: hueco en la
lectura
```

- La **respuesta** es la casilla más grande de toda la tabla.
- Para saber **cómo** quedó el alineamiento, se hace el **retroceso**: desde esa casilla se vuelve paso a paso por el camino que la produjo, hasta llegar a un 0.

## LAS DOS PROPIEDADES QUE HAY QUE SABER DECIR

1. **Subestructura óptima:** la mejor solución grande está hecha de mejores soluciones pequeñas. Si el mejor alineamiento que termina en (i, j) viene de la diagonal, lo que queda antes es el mejor alineamiento que termina en (i−1, j−1). Si no lo fuera, se podría cambiar por uno mejor y el total mejoraría, lo cual es una contradicción (argumento de "cortar y pegar").
2. **Subproblemas superpuestos:** las mismas casillas se necesitan muchas veces. H[i−1][j−1] la usan tres casillas distintas. Una recursión sin tabla repetiría ese cálculo una cantidad **exponencial** de veces (≥ 3^min(m,n)). La tabla lo calcula **una sola vez**.

Esas dos propiedades son las que justifican usar **programación dinámica**.

### COMPLEJIDAD

- Hay m·n casillas y cada una cuesta O(1), así que el tiempo es **Θ(m·n) siempre** (mejor, peor y promedio son iguales: la tabla se llena completa sin importar las letras).
- Memoria Θ(m·n) por la tabla. Si solo quisiéramos el puntaje bastarían 2 filas, pero necesitamos el alineamiento para el retroceso.
- **Por qué importan las semillas:** contra la región candidata (≈ 70 letras) una lectura cuesta 50 × 70 = 3.500 casillas. Contra un genoma de un millón de letras costaría 50 millones.

### CÓMO SABEMOS QUE ESTÁ BIEN PROGRAMADO

- Los tests comparan contra una **fuerza bruta** que prueba todas las combinaciones posibles en cadenas pequeñas.
- Usan un **ejemplo clásico de los libros** (puntaje 13).
- Comprueban que el alineamiento devuelto suma exactamente el puntaje reportado.

Archivos: `c3_smith_waterman/smith_waterman.py`, `tests/test_c3_smith_waterman.py`.

---

## 8. C4: Quicksort aleatorizado (otra forma de ordenar)

### PARA QUÉ LO USAMOS

1. Para **comparar** con merge sort (C1), como pide el enunciado.
2. En C5, para ordenar cada pedazo en memoria. Quicksort ordena **en el mismo arreglo**, sin listas auxiliares, y eso importa cuando la memoria está limitada.

### CÓMO FUNCIONA

1. Se elige un elemento **al azar**: el **pivote**.
2. Se reacomoda el arreglo en **tres zonas**: `[ menores | iguales | mayores ]`. Esta es la **partición de 3 vías**.
3. Se repite lo mismo **solo** con la zona de menores y la de mayores. La zona de iguales ya quedó en su lugar.

```
[5 3 8 3 1 9 3]   pivote al azar = 3
[1 | 3 3 3 | 5 8 9]   → falta ordenar [1] y [5 8 9]
```

**¿Por qué 3 zonas?** Porque los k-mers se repiten mucho. Con 2 zonas, los iguales se volverían a revisar una y otra vez. Con 3 zonas, si todos los elementos son iguales, basta una sola pasada: Θ(n).

**¿Por qué al azar?** Si el pivote fuera siempre el primero, una lista **ya ordenada** daría siempre el peor pivote (el mínimo), y el algoritmo tardaría $n^2/2$. Al azar, **ninguna entrada** es mala de forma sistemática.

## CUÁNTAS COMPARACIONES HACE (CASO ESPERADO)

El análisis clave, en palabras:

- Dos elementos se comparan **solo si uno de ellos fue pivote** mientras estaban juntos en la misma zona.
- Si en orden son el i-ésimo y el j-ésimo, se comparan con probabilidad **2/(j − i + 1)**: de los j − i + 1 elementos entre ellos, alguno de los dos tiene que ser el primero en salir como pivote.
- Se suman esas probabilidades para todos los pares (linealidad de la esperanza):

$$E[\text{comparaciones}] = 2(n+1)\cdot H_n - 4n \approx 2\cdot n\cdot \ln n \approx 1{,}39 \cdot n\cdot \log_2 n$$

($H_n = 1 + 1/2 + \ldots + 1/n \approx \ln n$).

> **Ojo:** la esperanza es sobre los **pivotes aleatorios**, no sobre los datos. Vale para **cualquier** entrada, incluso una ya ordenada.

| Caso | Cuándo pasa | Costo |
|---|---|---|
| Mejor | el pivote siempre es la mediana; o todos los elementos iguales | Θ(n log n); Θ(n) si son todos iguales |
| Esperado | siempre, para cualquier entrada | Θ(n log n), ≈ 1,39·n·$\log_2$ n comparaciones |
| Peor | el pivote siempre es el mínimo o el máximo (probabilidad casi cero) | Θ($n^2$) |

El 1,39 es el valor al que se acerca cuando n es enorme. Con n = 500.000 la fórmula exacta da 1,23 (el −4n todavía pesa), y eso es justo lo que medimos.

**Comparado con merge sort:** quicksort hace **más** comparaciones (≈ 1,23 contra ≈ 0,93 veces n·$\log_2$ n en nuestros tamaños). Pero no necesita listas extra, y en otros lenguajes suele ser más rápido por el uso de la caché.

Archivos: `c4_quicksort/quicksort.py`, `tests/test_c4_quicksort.py`.

---

# 9. C5: Ordenamiento externo (cuando los datos no caben en memoria)

### EL PROBLEMA

Imaginen que hay que ordenar un millón de fichas, pero en el escritorio solo caben **M** a la vez. El resto está en un archivador (el **disco**), y lo único que se puede hacer es traer o devolver **una carpeta de B fichas** a la vez (un **bloque**).

Ir al archivador es **lentísimo** comparado con trabajar en el escritorio. Por eso aquí **no se cuenta el tiempo de CPU**: se cuentan las **transferencias de bloques** (E/S = entradas/salidas).

### EL DISCO SIMULADO

La clase `DiscoSimulado` es ese archivador:

- guarda los registros en un archivo binario, en bloques de B registros;

- cada registro mide 12 bytes: el k-mer (8 bytes) más la posición (4 bytes);
- solo se puede acceder con `leer_bloque(i)` y `escribir_bloque(i, datos)`, y **cada llamada se cuenta**;
- el enunciado **prohíbe** leer el archivo de cualquier otra forma.

## EL ALGORITMO, EN DOS FASES

**Fase 1, hacer corridas:** traer M registros al escritorio, ordenarlos con quicksort (C4) y devolverlos como una **corrida** (un tramo ordenado). Al final hay N/M corridas ordenadas.

**Fase 2, mezclar:** igual que en merge sort, pero mezclando **muchas corridas a la vez**.

- En el escritorio caben M/B carpetas. Se usa **una carpeta por corrida** más **una de salida**, así que se pueden mezclar **k = M/B − 1 corridas a la vez**.
- Para saber cuál es el registro más pequeño entre las k corridas se usa un **min-heap** (implementado por nosotros).
- Cada pasada divide el número de corridas por k, y se repite hasta que queda una sola.

## LA FÓRMULA DE E/S

- **Cada pasada** (la fase 1 o una de mezcla) lee todos los bloques una vez y los escribe una vez: **2·N/B** E/S.
- Hay **1** pasada de la fase 1 y **⌈log_k(N/M)⌉** pasadas de mezcla.

```
E/S = 2 · (N/B) · (1 + ⌈log_(M/B − 1)(N/M)⌉)
```

**Ejemplo:** N = 1.000.000, B = 100, M = 10.000.

- k = 99 y hay 100 corridas, así que se necesitan **2** pasadas de mezcla (100 → 2 → 1).
- E/S = 2 · 10.000 · 3 = **60.000**.
- El contador dio **exactamente** 60.000.

**Memoria:** el código lleva la cuenta de cuántos registros hay en memoria y **falla** si alguna vez supera M. Así se demuestra que se respeta el límite.

Archivos: `comun/disco_simulado.py`, `c5_external_sort/external_sort.py`, `c5_external_sort/heap_min.py`, `tests/test_c5_external_sort.py`.

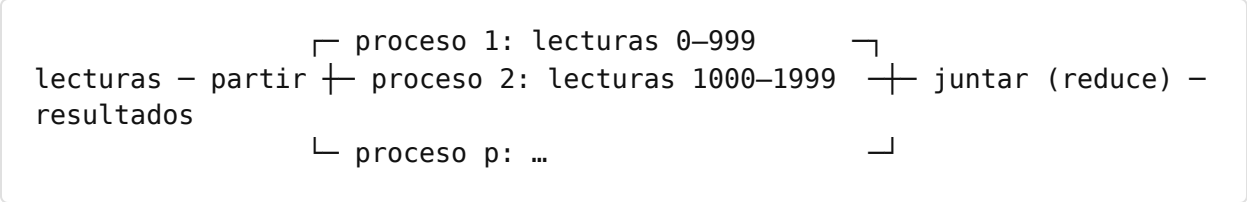
---

# 10. C6: MapReduce (alinear en paralelo)

## LA IDEA

Cada lectura se alinea **sin depender de las demás**. Entonces se pueden repartir entre **p procesos** que trabajan al mismo tiempo, como repartir exámenes entre varios profesores para calificar.

- **Map:** cada proceso alinea su paquete de lecturas.
- **Reduce:** se juntan los resultados en orden y se calculan estadísticas.




El índice se construye **una vez**, antes de repartir. En Linux los procesos hijos lo **heredan** sin copiarlo (`fork`).

## TRABAJO, PROFUNDIDAD Y SPEEDUP

| Concepto | Qué significa | En nuestro caso |
|---|---|---|
| **Trabajo W** | tiempo total si lo hiciera **un** solo procesador | índice + R lecturas × (costo de una) + reduce |
| **Profundidad D** | tiempo con **infinitos** procesadores (la cadena que no se puede paralelizar) | índice + costo de **una** lectura + reduce |
| **Speedup S(p)** | cuántas veces más rápido con p procesos: $T_1$ / T_p | lo medimos con p = 1, 2, 4, 8 y 16 |

### LEY DE AMDAHL

Si una fracción **f** del programa es **serial** (no se puede repartir):

```
S(p) = 1 / ( f + (1 – f)/p )     y nunca más que 1/f
```

**Ejemplo:** si construir el índice es el 40 % del tiempo (f = 0,4), aunque tengamos 1000 procesadores el programa nunca será más de 1/0,4 = **2,5 veces** más rápido.

Por eso medimos **dos escenarios**:

1. **Solo alineamiento**, con el índice ya construido: f pequeña, y el speedup se acerca a p.
2. **Sistema completo**, incluyendo construir el índice: f grande, y el speedup se frena rápido.

**Otro límite real:** la máquina de pruebas tiene **8 núcleos físicos** (16 “hilos”). Pasar de 8 a 16 procesos casi no ayuda, porque los hilos comparten el mismo núcleo.

**Costo de comunicación:** las lecturas tienen que viajar a los procesos y los resultados volver. Eso suma tiempo que no existe en la versión secuencial.

Archivos: `c6_mapreduce/mapreduce.py`, `tests/test_c6_mapreduce.py`.

---

# 11. Los experimentos: comprobar que la teoría se cumple

### LAS REGLAS DEL ENUNCIADO (Y CÓMO LAS CUMPLIMOS)

- Medir con `time.perf_counter()`.
- Repetir cada medición **5 veces** y reportar la **mediana**. La mediana es mejor que el promedio porque un pico raro (el sistema operativo haciendo otra cosa) no la mueve.
- Generar los datos **antes** de empezar a medir.
- Usar al menos 5 tamaños, **duplicando** n cada vez.

Todo eso lo hace `experimentos/bench.py`.

## CÓMO LEER UNA GRÁFICA LOG-LOG (LO MÁS PREGUNTADO)

En una gráfica log-log ambos ejes van en escala logarítmica (1, 10, 100, 1000…). La gracia es que:

> Si $T(n) \approx c \cdot n^p$, en log-log se ve una **recta con pendiente p**.

| **Si la pendiente da…** | **el algoritmo es…** |
|---|---|
| ≈ 1 | lineal, $\Theta(n)$ |
| ≈ 1,1 (en nuestros tamaños) | n log n |
| ≈ 2 | cuadrático, $\Theta(n^2)$ |

**Ojo con n log n:** no es exactamente una recta en log-log. En el rango que medimos ($10^3$ a $5 \cdot 10^5$) su pendiente "esperada" es **≈ 1,10**, no 1. Por eso, para n log n, el enunciado pide además graficar **T(n) / (n log n)**: si la teoría es correcta, esa división da **más o menos una constante** (una línea plana).

## RESULTADOS QUE OBTUVIMOS

| Componente | Pendiente medida | Esperada | Veredicto |
|---|---|---|---|
| C1 merge sort | **1,12** | ≈ 1,10 | ✅ |
| C2 arreglo dinámico | **1,01** | 1 | ✅ (además costo < 3n y alcancía nunca negativa) |
| C3 con lectura fija (m = 50) | **1,02** | 1 | ✅ |
| C3 con m = n | **2,08** | 2 | ✅ |
| C4 quicksort | **1,14** | ≈ 1,10 | ✅ (comparaciones = 0,99–1,03 veces las esperadas) |
| C5 ordenamiento externo | E/S contadas = predichas en **10 de 10** casos | igualdad | ✅ (exacto, con 1 a 6 pasadas) |
| C6 trabajo vs. lecturas (p = 1) | **1,01** | 1 | ✅ |
| C6 speedup con 8 procesos | **5,27** | 7,64 según Amdahl con f medida | ❌ explicado abajo |

**Diferencias que hay que saber explicar:**

- La teoría habla de **cómo crece** el tiempo, no de cuántos segundos tarda. Python es unas 100 veces más lento que C, pero eso solo cambia la constante c, **no la pendiente**.
- Con n muy grande, el tiempo por elemento sube un poco (≈ 13 % en C1). La causa es la **memoria caché**: los datos ya no caben en la memoria rápida del procesador. No es un error del algoritmo.
- En C2 el costo dio ≈ 2·n y no 3·n porque con $n = 1000 \cdot 2^i$ el arreglo estaba justo antes de duplicar. Con $n = 2^j + 1$ (justo después de duplicar) da casi 3·n, que es exactamente el peor caso que predice el método contable.

- **C6, el speedup fue menor de lo que predijo Amdahl.** Medimos directamente la parte serial (crear los procesos y el reduce): apenas f = 0,7 %. Con eso, Amdahl predecía 7,6× con 8 procesos, y medimos 5,3×.
  - **Qué lo explica:** la **métrica de Karp-Flatt** calcula, a partir del speedup medido, cuánta parte serial "efectiva" hubo. Dio ≈ 7 %, casi igual con 2, 4 y 8 procesos. Es decir, hay un sobrecosto fijo de ≈ 0,6 s que se comporta como si fuera serial. Con ese f, Amdahl acierta.
  - **De dónde sale el sobrecosto (explicación más probable):** cada proceso hijo hereda el índice, pero Python escribe un contador en cada objeto que lee. Eso obliga a cada hijo a **copiar** las páginas de memoria del índice (*copy-on-write*), y esa copia no se reparte entre los procesos.
  - **Con 16 procesos empeora:** la máquina solo tiene 8 núcleos de verdad.
- **C6, sistema completo:** construir el índice es serial y es ≈ 34 % del tiempo, así que Amdahl dice que **nunca** pasará de ≈ 3× más rápido. Medimos 2,2× con 8 procesos, consistente con ese límite.

Las gráficas están en `experimentos/graficas/` y los números en `experimentos/resultados/`.

---

# 12. Cómo correr todo

Desde la carpeta del repositorio:

```
pytest -q                                        # todas las pruebas
(deben pasar 134)
python -m datos.generar 10000 100 42             # genera genoma y
lecturas
python -m experimentos.exp_c1_mergesort          # experimento C1 (~1
min)
python -m experimentos.exp_c2_arreglo_dinamico   # experimento C2
python -m experimentos.exp_c3_smith_waterman     # experimento C3
python -m experimentos.exp_c4_quicksort          # experimento C4 (~2
min)
python -m experimentos.exp_c5_external_sort      # experimento C5 (~5
min)
python -m experimentos.exp_c6_mapreduce          # experimento C6 (~5
min, usa todos los núcleos)
```

### MAPA DEL REPOSITORIO

| Carpeta | Qué hay |
|---|---|
| `datos/` | generador de genoma y lecturas |
| `comun/` | k-mers en números, índice, alineador, disco simulado |
| `c1_mergesort/` … `c6_mapreduce/` | los seis componentes |
| `tests/` | las pruebas |
| `experimentos/` | mediciones, gráficas y resultados |
| `docs/` | enunciado, análisis teórico, plan de experimentos, informe y esta guía; `docs/pdf/` con los PDF |

**Regla del enunciado:** en el núcleo de los componentes **no se puede usar** `sorted`, `.sort()` ni `heapq`. Solo se usan en los tests, para comparar resultados.

---

# 13. Preguntas probables de la sustentación (con respuesta corta)

### C1, merge sort

1. *¿Cuál es la recurrencia y de dónde sale?* T(n) = 2T(n/2) + Θ(n): dos llamadas sobre mitades más una mezcla lineal.
2. *¿Qué caso del Teorema Maestro aplica?* Caso 2: a = 2, b = 2, f(n) = n = n^($\log_2$ 2). Resultado: Θ(n log n).
3. *¿Por qué no se puede ordenar comparando en menos de n log n?* Árbol de decisión: necesita n! hojas, así que la altura es ≥ $\log_2$ n! ≈ n log n.
4. *¿Qué entrada da el mejor caso de comparaciones?* La ya ordenada: (n/2)·$\log_2$ n.
5. *¿Es estable?* Sí, porque `mezclar` usa `<=` y así los iguales conservan su orden.

### C2, arreglo dinámico

1. *¿Por qué 3 créditos?* 1 paga la escritura y 2 se ahorran para copiar ese elemento y otro más viejo en la siguiente duplicación.
2. *¿Amortizado es lo mismo que promedio?* No. Amortizado es el reparto del costo sobre una **secuencia** de operaciones en el peor caso; no depende de datos al azar.
3. *¿Qué pasa si en vez de duplicar se suma un tamaño fijo (+10)?* Cada crecimiento copia todo, el costo total sale cuadrático y append ya no es O(1) amortizado.

**C3, Smith-Waterman**

1. *¿Qué significa H[i][j]?* El mejor puntaje de un alineamiento que termina en la letra i de la lectura y la j de la región.
2. *¿Por qué hay un 0 en el máximo?* Para poder “empezar de nuevo”: eso es lo que lo hace **local**.
3. *Subestructura óptima y subproblemas superpuestos:* ver la sección 7. Hay que saber decirlo con palabras.
4. *¿Complejidad?* Θ(m·n) en tiempo y memoria, igual en todos los casos.
5. *¿Por qué no alinear contra el genoma completo?* Costaría m·n con n gigante; las semillas reducen la región a ≈ 70 letras.

**C4, quicksort aleatorizado**

1. *¿Por qué el pivote al azar?* Para que ninguna entrada sea mala de forma sistemática. Con pivote fijo, una lista ya ordenada cuesta $n^2/2$.
2. *¿De dónde sale 2n ln n?* $z_i$ y $z_j$ se comparan con probabilidad $2/(j-i+1)$; se suma sobre todos los pares (linealidad de la esperanza).
3. *¿El “caso esperado” supone datos al azar?* No. Es sobre los pivotes aleatorios: vale para **cualquier** entrada.
4. *¿Para qué sirve la partición de 3 vías?* Con k-mers repetidos, los iguales quedan fijos de una vez. Si todos son iguales, cuesta Θ(n).
5. *¿Por qué hace más comparaciones que merge sort y aun así se usa?* Ordena en su lugar (memoria O(log n)), y eso es justo lo que necesita C5.

**C5, ordenamiento externo**

1. *¿Qué se cuenta y por qué no el tiempo?* Las transferencias de bloque: en memoria externa, acceder al disco domina todo lo demás.

2. *¿Por qué k = M/B − 1?* Caben M/B bloques en memoria: uno por corrida y uno para la salida.
3. *¿De dónde sale la fórmula?* Cada pasada lee y escribe todo (2N/B). Hay 1 pasada para crear corridas y ⌈log_k(N/M)⌉ de mezcla.
4. *¿Cómo garantizan que no usan más de M?* El código lleva la cuenta de los registros en memoria y falla si supera M. Los tests lo verifican.
5. *¿Por qué a veces no coincide exacto?* Solo si N no es múltiplo de B: el último bloque está incompleto.

**C6, MapReduce**

1. *¿Qué es trabajo y qué es profundidad?* El tiempo con 1 procesador y el tiempo con infinitos procesadores.
2. *¿Qué dice Amdahl?* Con fracción serial f, el speedup nunca supera 1/f, por más procesadores que haya.
3. *¿Qué es lo serial en nuestro sistema?* Construir el índice, crear y cerrar los procesos, y el reduce.
4. *¿Por qué no mejora de 8 a 16 procesos?* La máquina tiene 8 núcleos físicos; los otros 8 “hilos” comparten núcleo.
5. *¿Por qué el resultado paralelo es igual al secuencial?* Cada lectura se alinea de forma independiente, y el reduce junta los bloques en el mismo orden. Hay un test que lo comprueba.

**Experimentos**

1. *¿Cómo se lee la pendiente log-log?* Si $T \approx c \cdot n^p$, la pendiente es p.
2. *¿Por qué mediana y no promedio?* Porque es robusta a mediciones raras.
3. *¿Por qué la teoría no predice los segundos exactos?* Porque las constantes dependen del intérprete de Python, la caché y la máquina. La teoría predice el **crecimiento**.

---

# 14. Glosario

| Término | Significado sencillo |
|---|---|
| Genoma | el “libro” de letras A, C, G, T |
| Lectura | pedacito del genoma (50 letras) copiado con errores |
| k-mer | k letras seguidas |
| Semilla | k-mer de la lectura que también aparece en el genoma: una pista de dónde está |
| Región candidata | zona del genoma donde probablemente está la lectura |
| Alinear | acomodar la lectura sobre el genoma, columna por columna |
| Indel | inserción o borrado de una letra |
| Divide y vencerás | partir en subproblemas, resolverlos y combinar |
| Programación dinámica | guardar en una tabla las respuestas de subproblemas para no repetirlas |
| Amortizado | costo repartido sobre una secuencia de operaciones |
| Semilla fija (random) | número que hace que lo “aleatorio” salga siempre igual |
| Log-log | gráfica con ambos ejes logarítmicos; las potencias se ven como rectas |
| Pivote | elemento que se usa para partir el arreglo en quicksort |
| Bloque | grupo de B registros que se lee o escribe de una vez en el disco |
| Corrida | tramo ordenado de datos guardado en el disco |
| E/S | entradas/salidas: lecturas y escrituras de bloques |
| Min-heap | árbol que permite sacar siempre el elemento más pequeño en O(log k) |
| Speedup | cuántas veces más rápido corre con p procesos que con 1 |
| Fracción serial f | parte del programa que no se puede repartir entre procesos |